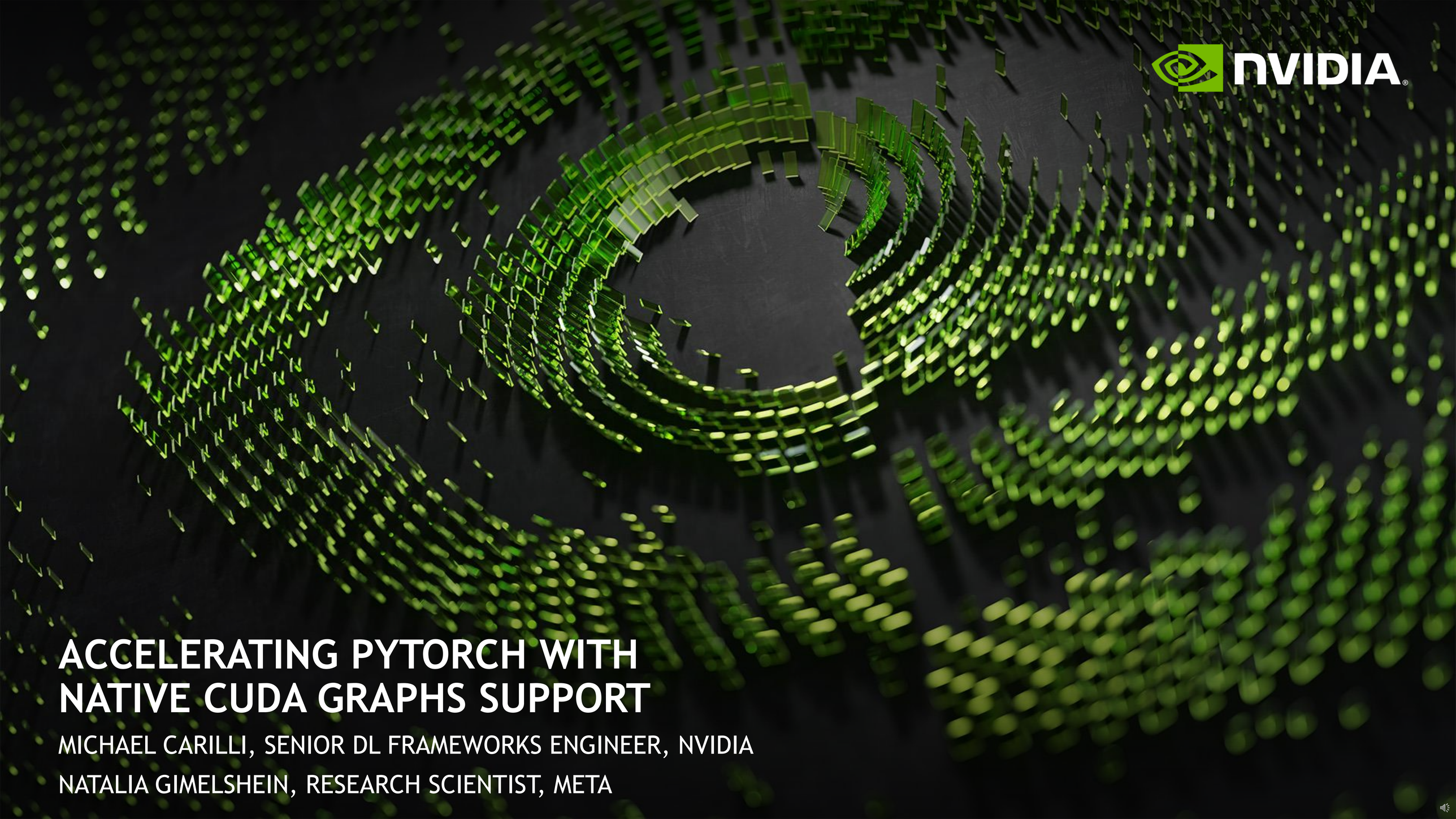


ACCELERATING PYTORCH WITH NATIVE CUDA GRAPHS SUPPORT

MICHAEL CARILLI, SENIOR DL FRAMEWORKS ENGINEER, NVIDIA

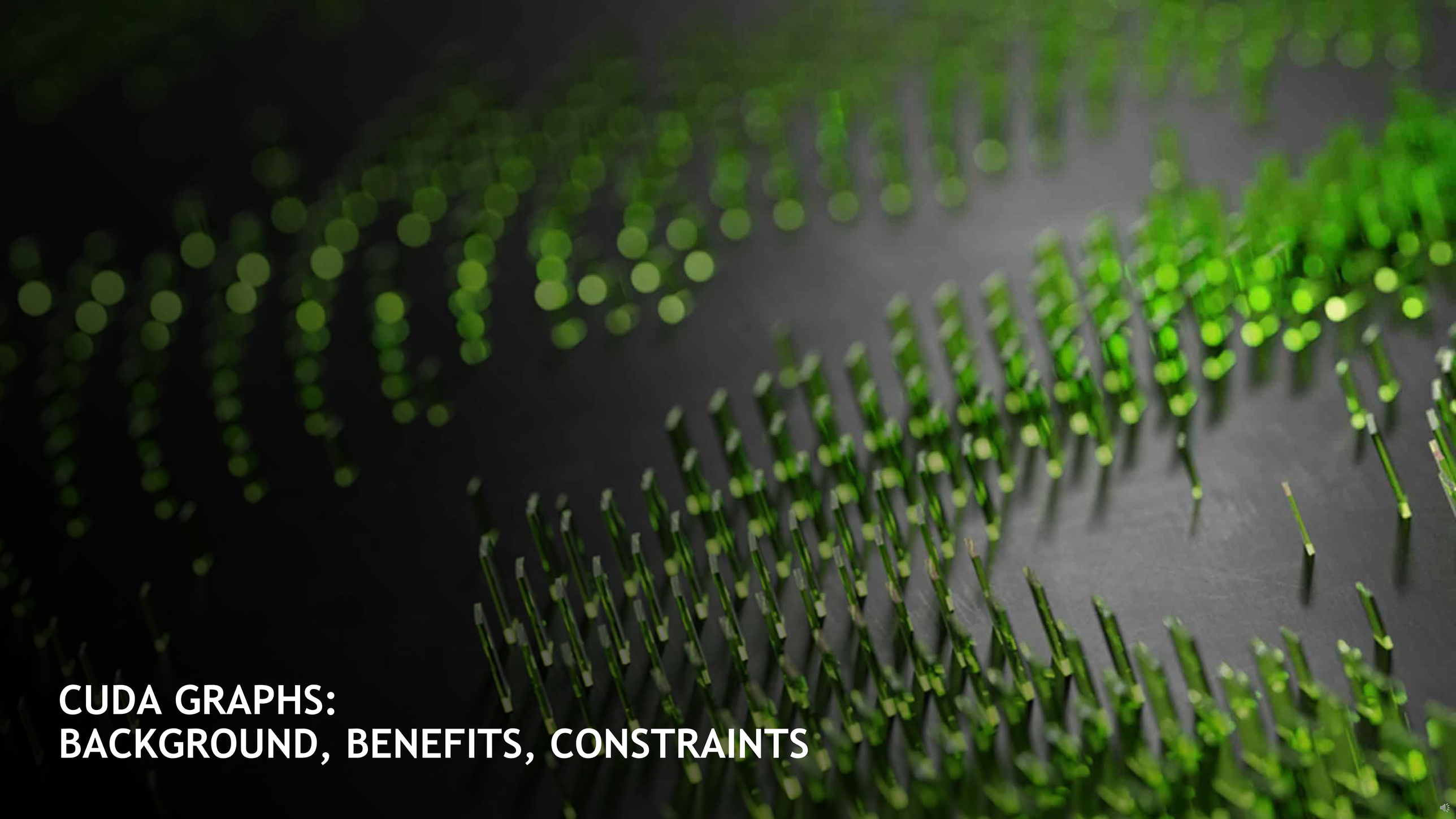
NATALIA GIMELSHEIN, RESEARCH SCIENTIST, META

AGENDA

CUDA Graphs: Background, Benefits, Constraints

Pytorch API

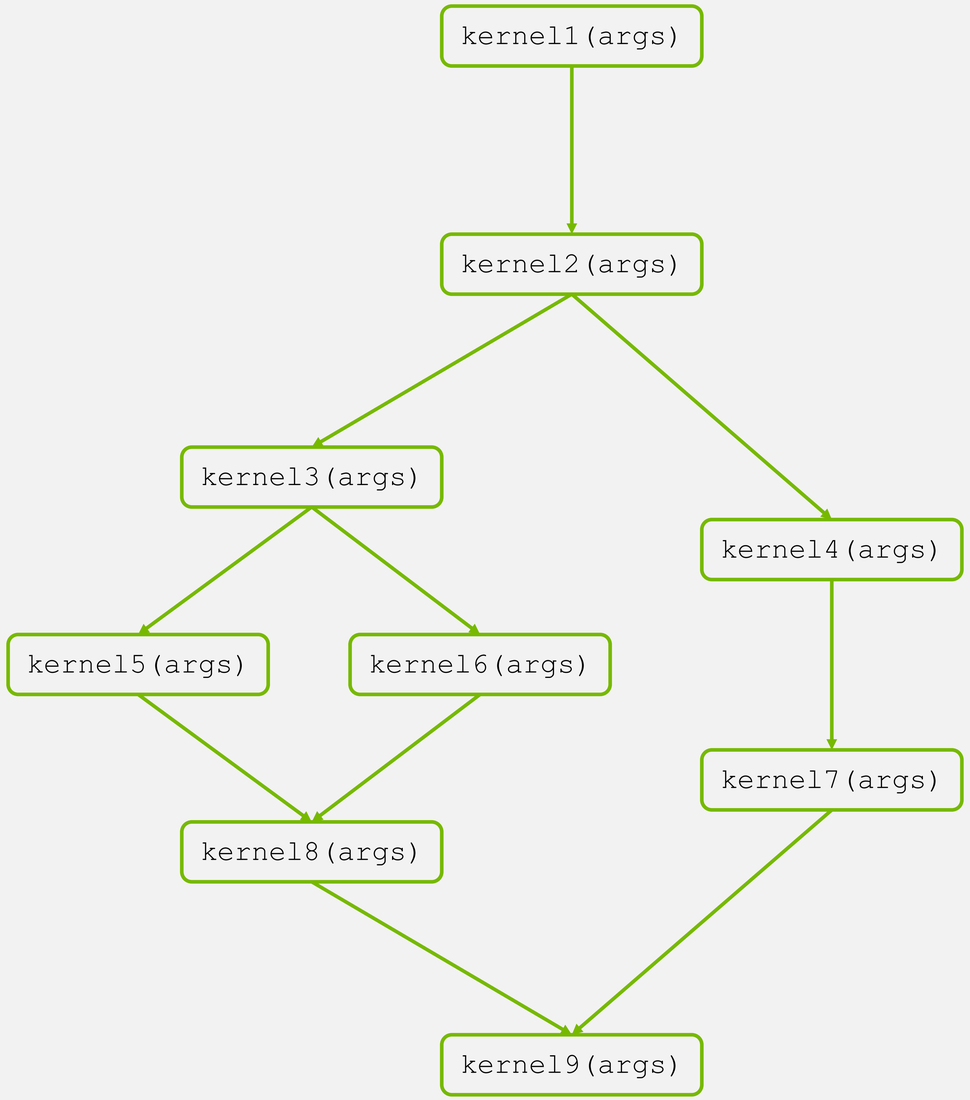
Usage with `DistributedDataParallel`, `torch.cuda.amp`, JIT



CUDA GRAPHS:
BACKGROUND, BENEFITS, CONSTRAINTS

BACKGROUND

- A CUDA Graph is a static record of GPU work.
- Created once
- Replayable with a single CUDA API call, as many times as desired
- Each replay runs the same kernels, with the same arguments and dependencies, acting on (typically) the same memory addresses.



BENEFITS

Graph replay greatly reduces CPU overhead.

EAGER EXECUTION

- Enqueues GPU ops one at a time
- Each op incurs CPU overhead
  (from Python, PyTorch, CUDA driver).
- Often CPU limited
  (the CPU can’t feed the GPU fast enough)

GRAPH REPLAY

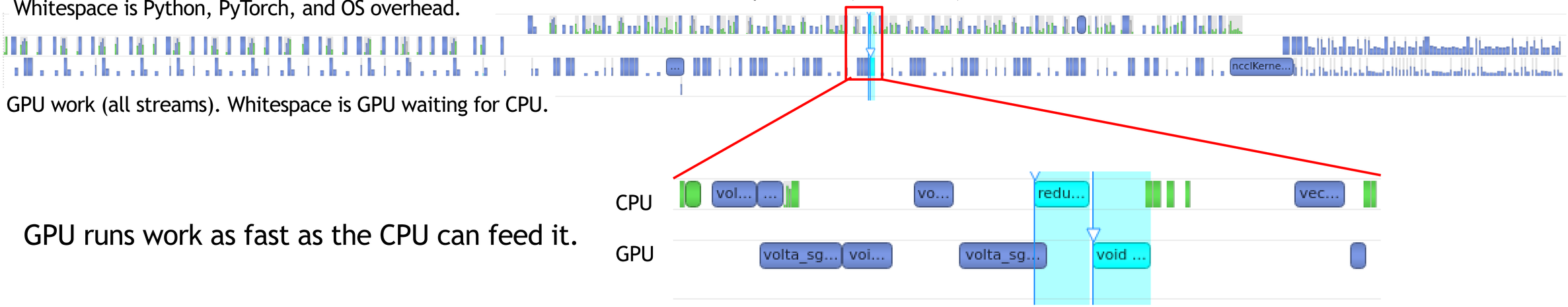
- Submits entire record of GPU ops with a single “cudaGraphLaunch” call.
- No longer CPU limited
- Improved GPU utilization
- Reduced end to end time

EXAMPLE CPU-LIMITED EAGER ITERATION (8.4 millisec)

CPU forward-pass work. Shows CUDA API calls.
Whitespace is Python, PyTorch, and OS overhead.

CPU backward-pass work (side thread)

Optimizer step



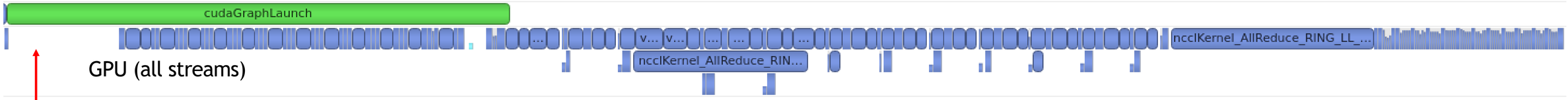
GPU work (all streams). Whitespace is GPU waiting for CPU.

GPU runs work as fast as the CPU can feed it.

GRAPH REPLAY (2.1 millisec)

CPU

CPU idle. Free time to load data, launch the next iteration's replay, etc.



GPU (all streams)

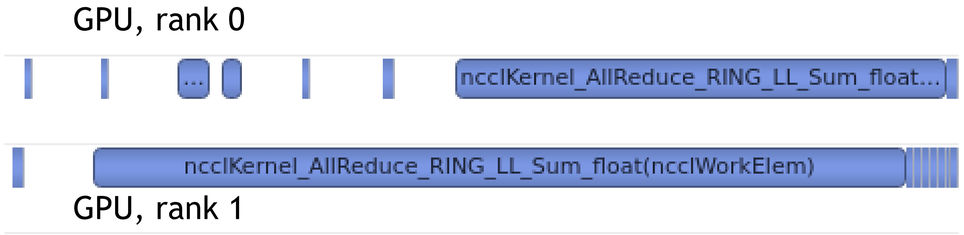
Gap here due to cold-start latency.
Would be hidden if this were the 2nd iteration or later.

BENEFITS AT SCALE

Graph replay reduces jitter (runtime imbalance across ranks).

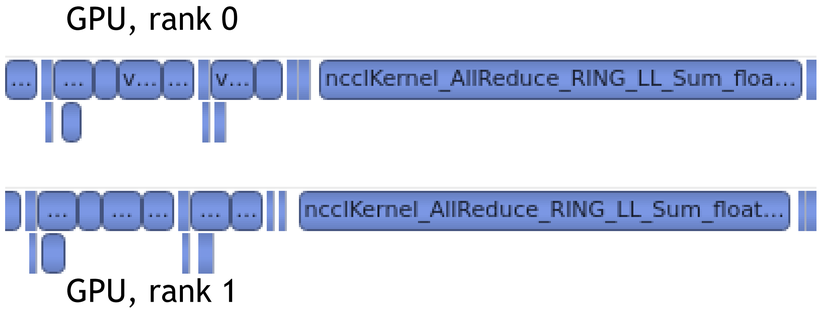
In typical distributed training, all ranks wait for the slowest. Jitter slows the entire job.

EAGER EXECUTION



- CPU-limited networks are often more vulnerable to jitter.

GRAPH REPLAY



- Without CPU overhead in the way, GPU timelines are more stable across ranks.
- Jitter is greatly reduced.

EAGER (END OF BACKWARD + OPTIMIZER STEP)

Allreduce kernels must sync across rank
Rank 0 is lagging.

Optimizer work

Rank 0 starts lagging again

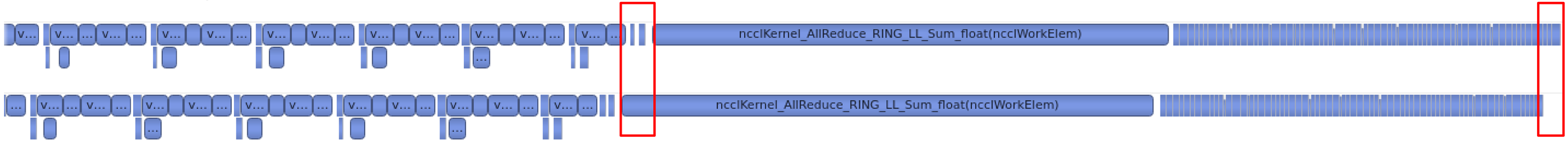
GPU backward-pass work on rank 0 (all streams)

GPU backward-pass work on rank 1 (all streams)

Rank 1 kernel spins, waiting for rank 0.

GRAPH REPLAY (SAME SECTION OF BACKWARD + OPTIMIZER STEP)

GPU backward-pass work on rank 0



GPU backward-pass work on rank 1

Relatively, much less jitter.

END-TO-END TRAINING SPEEDUPS WITH CUDA GRAPHS

1.1X

BERT
MLPERF
4096 A100s
BATCH 3/GPU

1.6X

MASK-RCNN
MLPERF
272 A100s
BATCH 1/GPU

2.2X

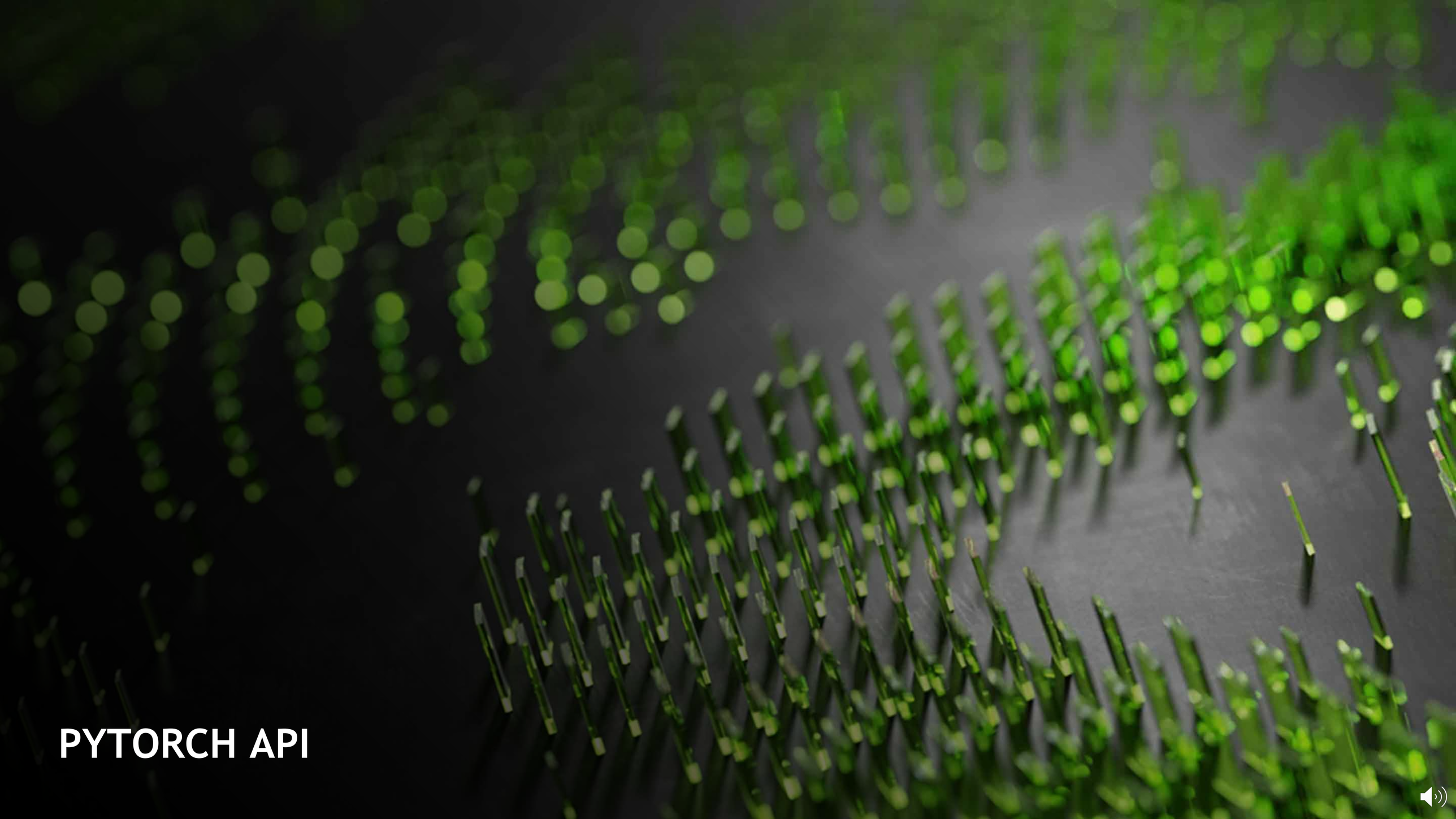
DLRM
NVIDIA DEEP LEARNING EXAMPLES
8 A100s
BATCH 512/GPU

CONSTRAINTS

A *capturable* op sequence obeys these constraints:

- No dynamic shapes (shapes may not change across iterations)
- No dynamic control flow
- No CPU<->GPU synchronization
- No essential CPU ops or CPU side effects
- Others, less common:

  https://pytorch.org/docs/master/notes/cuda.html#constraints


PYTORCH API

API SUMMARY

`torch.cuda.CUDAGraph`

Primitive wrapper that stores and replays a graph.

`torch.cuda.graph`

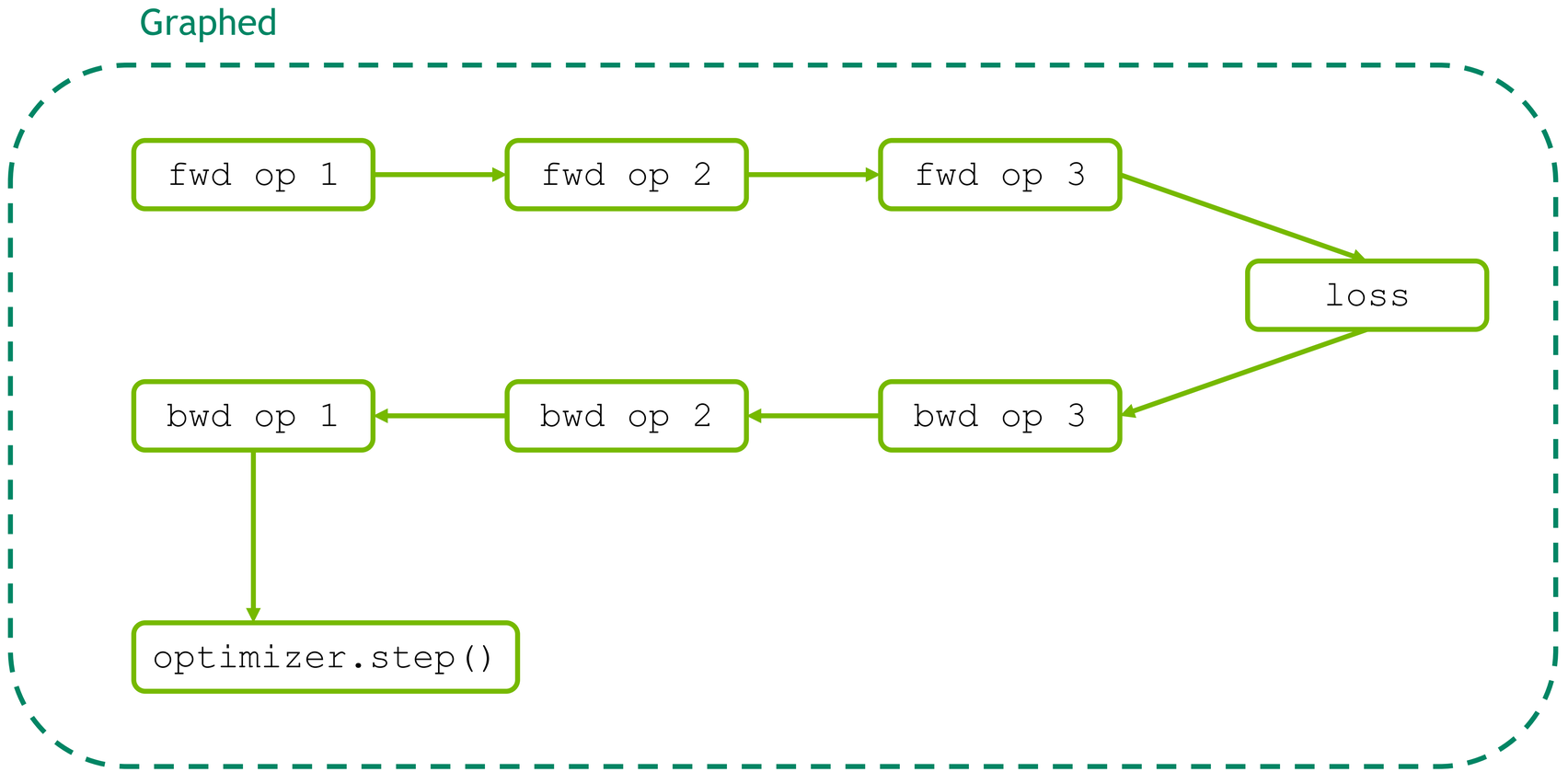
Context manager that graphs GPU work in its context.

`torch.cuda.make_graphed_callables`

Accepts callables and returns (autograd-aware) graphed versions.

WHOLE-NETWORK CAPTURE

If your entire network is capturable, use `torch.cuda.graph` to capture and replay an entire iteration.



- Warms up before capture
- Prevents capturing lazy-initialization work (like cudnn benchmarking) that isn’t needed every iteration
- Warmup must occur on a side stream.

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Placeholder inputs used for capture
- These will be the memory from which replays read input data.

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Constructs graph handle

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Sets grads to none before capture
- backward() will create .grad attributes with allocations from a graph-private memory pool.

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Captures an entire iteration
- During capture, no actual GPU work is performed, but GPU ops are recorded in the graph.

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Fills the graph's input memory with new data to compute on

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Runs the entire iteration (forward, backward, step)
- After replay(), params have been updated.
  capture_y_pred, capture_loss, and .grad attributes hold values from computing on this iteration's data.

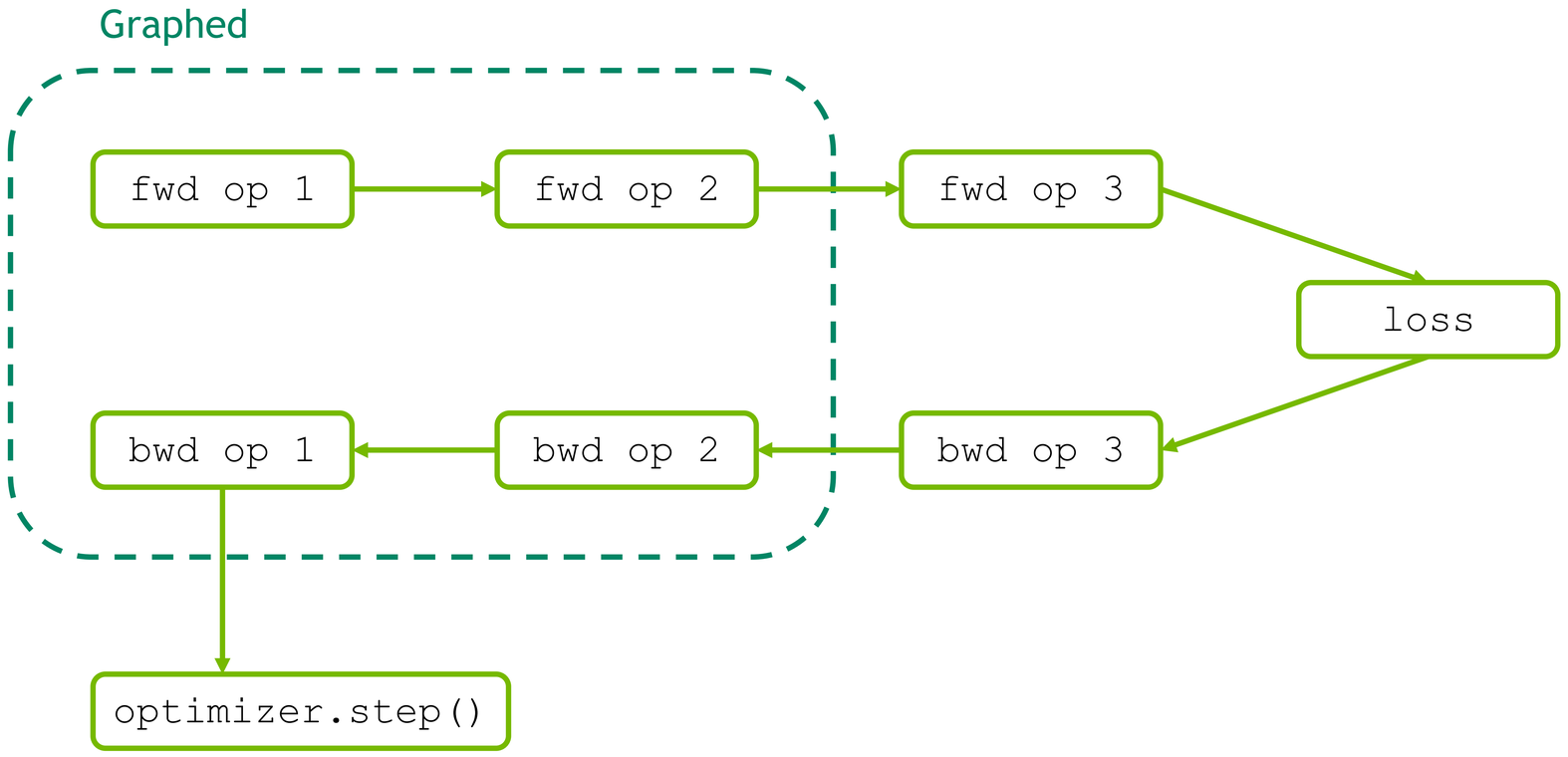
```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

- Don’t call optimizer.zero_grad() between iterations.
- The captured backward() refills static .grad tensors in place.
- In fact, calling optimizer.zero_grad(set_to_none=True) would be an error. You’d erase the .grad tensors the captured backward() expects to fill.

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

PARTIAL-NETWORK CAPTURE

If most (but not all) of your network is capturable,
use `torch.cuda.make_graphed_callables` to graph only those regions.



- item() incurs a CPU-GPU sync.
- Also, its result triggers data-dependent control flow.
- Whole-network capture is not possible.

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

- Sample inputs used for capture
- requires_grad state of sample inputs must match requires_grad state of real inputs to each callable.

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

- Creates autograd-aware graphed versions of graph-safe regions
- Explicit warmup not needed (make_graphed_callables handles warmup under the hood)

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

- Forward-pass work for each callable, if chosen, runs as a graph.
- Explicit copies to captured input tensors not needed (graphed callables handle this under the hood)

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

- Backward-pass work for each callable, if chosen, runs as a graph.

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

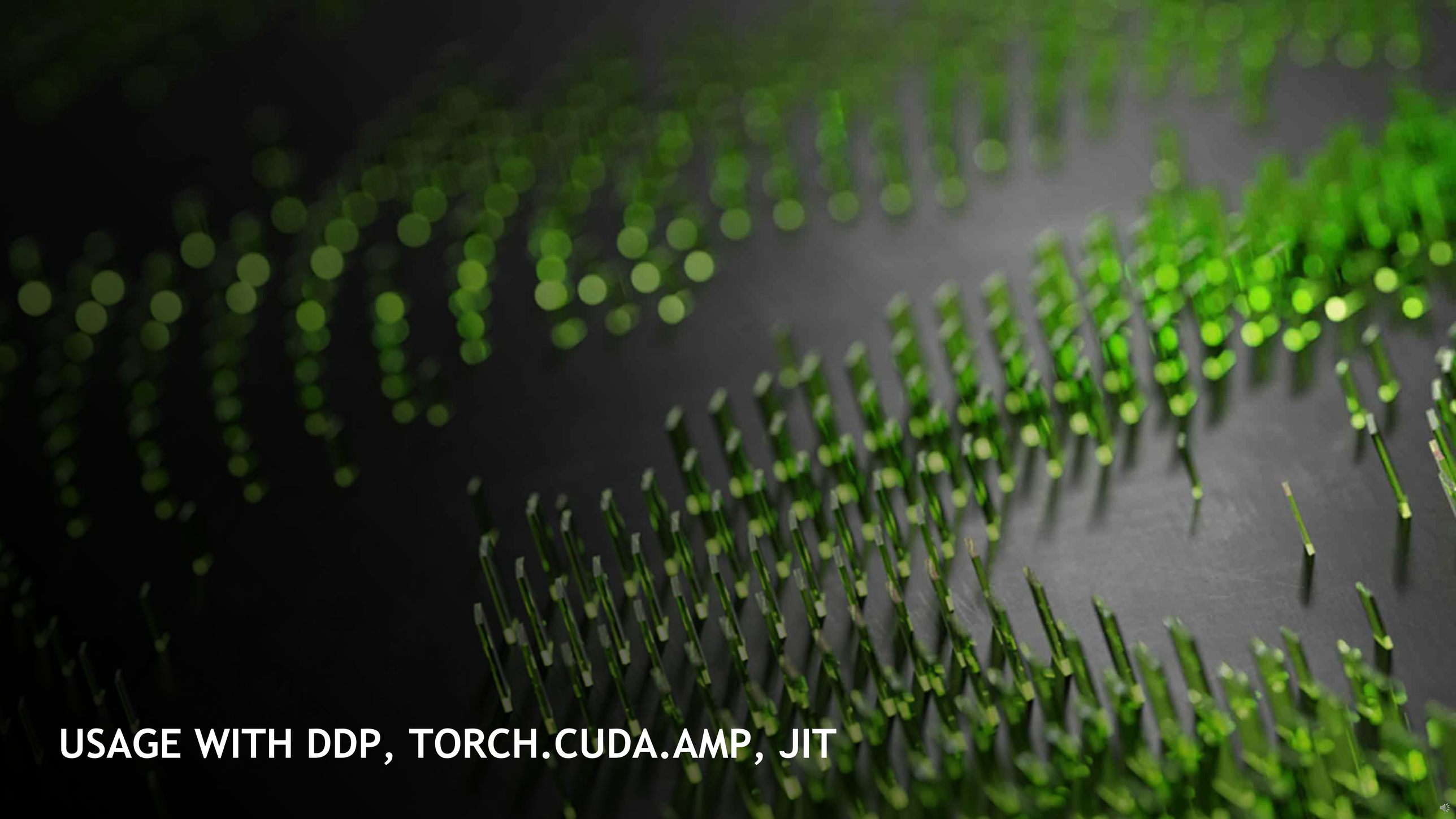
- You must call optimizer.zero_grad(set_to_none=True or False) between iterations as usual (unlike full-iteration capture).

```
x = torch.randn(shape1, device='cuda’)
h = torch.randn(shape2, device='cuda', requires_grad=True)

module1 = torch.cuda.make_graphed_callables(module1, (x,))
module2 = torch.cuda.make_graphed_callables(module2, (h,))
module3 = torch.cuda.make_graphed_callables(module3, (h,))

for data, target in zip(real_inputs, real_targets):
    optimizer.zero_grad(set_to_none=True)
    tmp = module1(data)
    if tmp.sum().item() > 0:
        tmp = module2(tmp)
    else:
        tmp = module3(tmp)
    loss = loss_fn(tmp, y)
    loss.backward()
    optimizer.step()
```

`graph` VS `make_graphed_callables`:

Whole-network capture with `graph` when you can

Partial-network capture with `make_graphed_callables` when you must


USAGE WITH DDP, TORCH.CUDA.AMP, JIT

USAGE WITH DISTRIBUTEDDATAPARALLEL

- https://pytorch.org/docs/master/notes/cuda.html#usage-with-distributeddataparallel

- With **`make_graphed_callables`**, DDP work is deferred outside graphed regions.
  - Call `make_graphed_callables` on a module BEFORE wrapping with DDP.
  - Otherwise, no changes needed
  - Advanced: Splitting a single graph-safe callable into several graphed callables can improve overlap of allreduces with backward work. This is probably more trouble than it's worth for most use cases.

- With **`graph`**, DDP work (i.e., allreduces) can occur within the graphed backward pass.
  To allow this, you must:
  - Use NCCL >= 2.9.6
  - Disable DDP's internal async error handling
  - Construct DDP in a side-stream context
  - Run at least 11 warmup iterations, to avoid some uncapturable work in DDP's internal logging

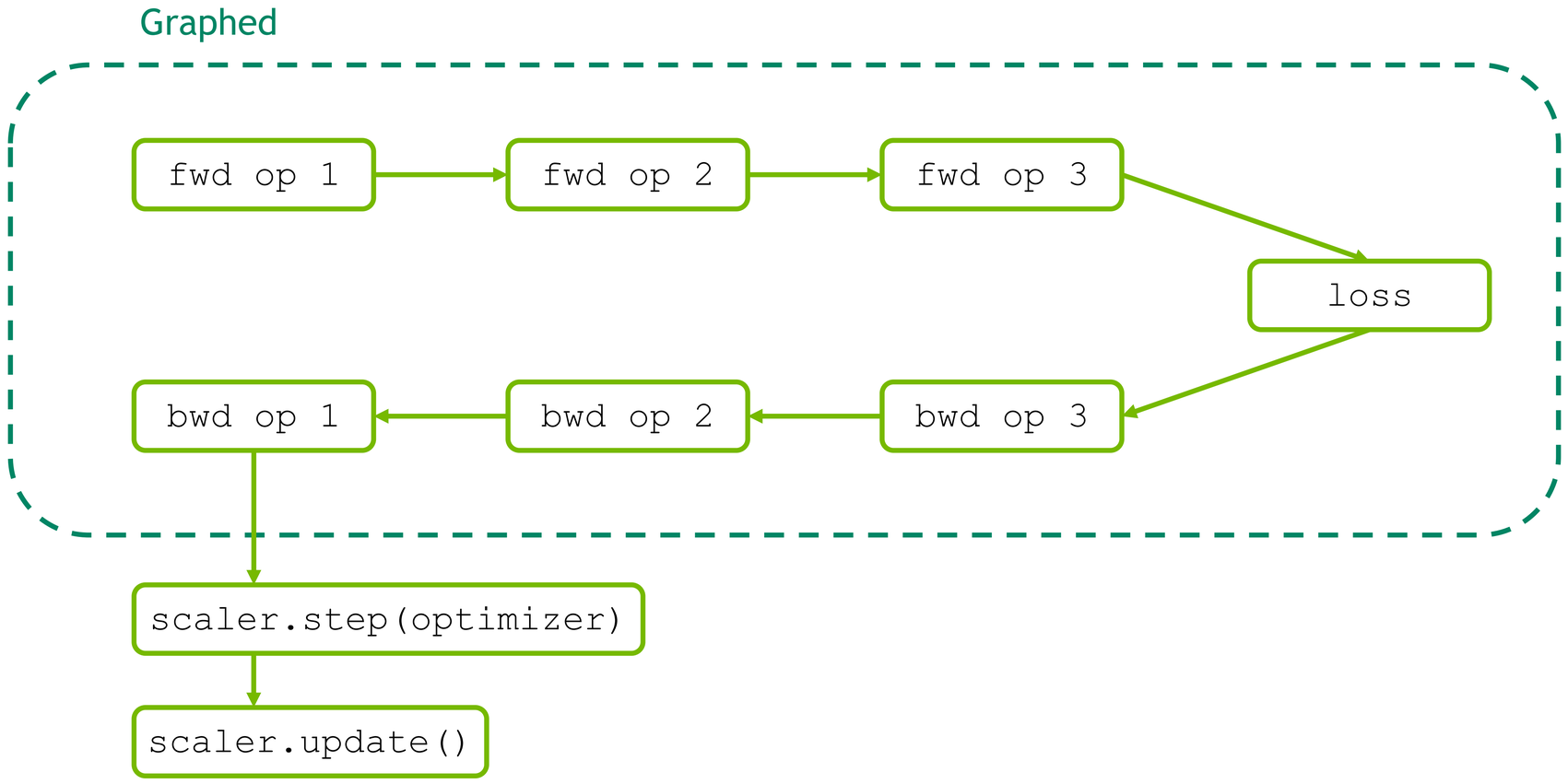
- Disables DDP's internal async error handling

- Constructs DDP in a side-stream context
- Runs 11 warmup iterations

```
os.environ["NCCL_ASYNC_ERROR_HANDLING"] = "0"
torch.distributed.init_process_group(…)

s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    model = torch.nn.parallel.DistributedDataParallel(model)

    for i in range(11):
        optimizer.zero_grad(set_to_none=True)
        y_pred = model(real_inputs[i])
        loss = loss_fn(y_pred, real_targets[i])
        loss.backward()
        optimizer.step()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    capture_y_pred = model(capture_input)
    capture_loss = loss_fn(capture_y_pred, capture_target)
    capture_loss.backward()
    optimizer.step()

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
```

USAGE WITH TORCH.CUDA.AMP

- https://pytorch.org/docs/master/notes/cuda.html#usage-with-torch-cuda-amp
- With partial-network capture via **`make_graphed_callables`**, no changes needed
- With full-network capture via **`torch.cuda.graph`**, `scaler.step` is not capturable for typical optimizers.
  - Don't capture `scaler.step` and `scaler.update`.



- Omits scaler.step and scaler.update during capture

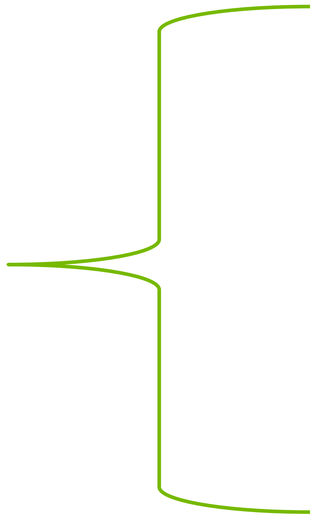
- Calls scaler.step and scaler.update eagerly after replay

```
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for i in range(3):
        optimizer.zero_grad(set_to_none=True)
        with torch.cuda.amp.autocast():
            y_pred = model(real_inputs[i])
            loss = loss_fn(y_pred, real_targets[i])
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
torch.cuda.current_stream().wait_stream(s)

capture_input = torch.empty(input_shape, device="cuda")
capture_target = torch.empty(target_shape, device="cuda")

g = torch.cuda.CUDAGraph()
optimizer.zero_grad(set_to_none=True)
with torch.cuda.graph(g):
    with torch.cuda.amp.autocast():
        capture_y_pred = model(capture_input)
        capture_loss = loss_fn(capture_y_pred, capture_target)
    scaler.scale(capture_loss).backward()
    # doesn't capture scaler.step or scaler.update

for data, target in zip(real_inputs, real_targets):
    capture_input.copy_(data)
    capture_target.copy_(target)
    g.replay()
    scaler.step(optimizer)
    scaler.update()
```

USAGE WITH JIT (EXPERIMENTAL)

- Use nvfuser backend.
- Trace or script your model.
- Run at least 3 warmup steps, then…
- …capture and replay the workload as you would without jit.

- Uses nvfuser backend
- Scripts the model
- Runs at least 3 warmup steps
- Captures and replays the workload as usual

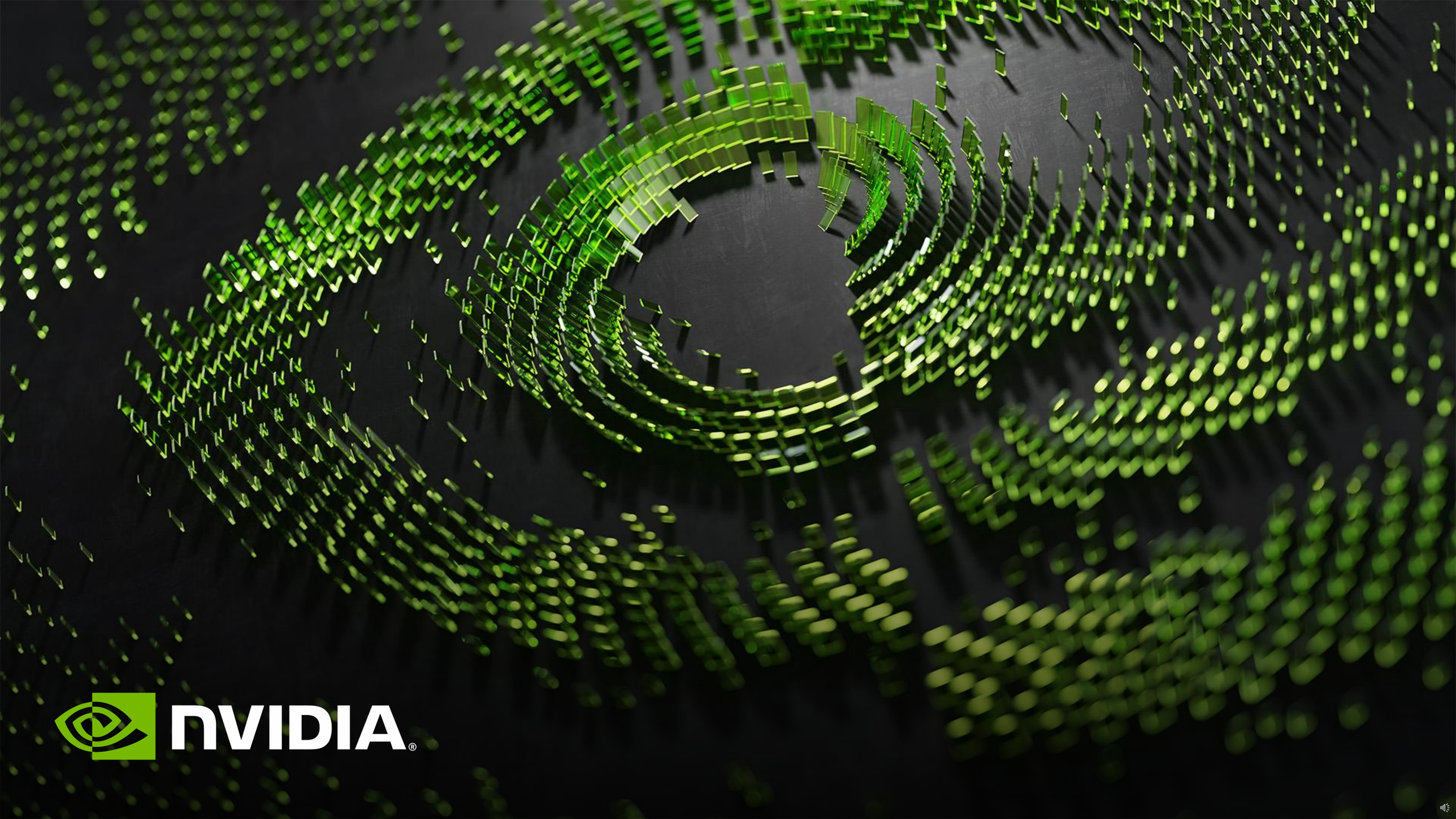


```
with torch.jit.fuser('fuser2'):
    # assumes model is already cuda
    model  = torch.jit.script(model)

    s = torch.cuda.Stream()
    s.wait_stream(torch.cuda.current_stream())
    with torch.cuda.stream(s):
        for i in range(3):
            optimizer.zero_grad(set_to_none=True)
            y_pred = model(real_inputs[i])
            loss = loss_fn(y_pred, real_targets[i])
            loss.backward()
            optimizer.step()
    torch.cuda.current_stream().wait_stream(s)

    capture_input = torch.empty(input_shape, device="cuda")
    capture_target = torch.empty(target_shape, device="cuda")

    g = torch.cuda.CUDAGraph()
    optimizer.zero_grad(set_to_none=True)
    with torch.cuda.graph(g):
        capture_y_pred = model(capture_input)
        capture_loss = loss_fn(capture_y_pred, capture_target)
        capture_loss.backward()
        optimizer.step()

    for data, target in zip(real_inputs, real_targets):
        capture_input.copy_(data)
        capture_target.copy_(target)
        g.replay()
```

DOCUMENTATION AND EXAMPLES

https://pytorch.org/docs/master/cuda.html#graphs-beta

https://pytorch.org/docs/master/notes/cuda.html#cuda-graphs


NVIDIA